

Relatório Técnico de Implementação: Sistema de Replicação de Arquivos

Luiz Henrique Murback Wiedmer¹, Bruno Corrado Crestani¹

¹Departamento de Informática – Universidade Federal do Paraná (UFPR)
Curitiba – PR – Brasil

lhmw@inf.ufpr.br, bcc22@inf.ufpr.br

Resumo. *Este relatório descreve a implementação de um sistema de replicação de arquivos baseado no modelo cliente-servidor. O sistema permite que clientes enviem arquivos para um servidor primário utilizando sockets TCP. O servidor primário é responsável pelo armazenamento e pela replicação do conteúdo para servidores espelhos (mirrors). Este documento detalha o protocolo de comunicação, o tratamento de mensagens, as decisões de projeto referentes ao gerenciamento de memória e concorrência e o modo de uso do sistema.*

1. Visão Geral do Sistema

O sistema é composto por três tipos de processos interconectados:

- **Cliente:** Conecta-se ao servidor primário e emite comandos como `upload`, `list`, `help`, `exit` e `endall`.
- **Servidor Primário:** Gerencia a comunicação com o cliente, realiza o armazenamento local dos arquivos e é o único responsável por iniciar o processo de replicação para as réplicas.
- **Servidores Réplica (Mirror):** Mantêm cópias dos arquivos replicados pelo servidor primário e respondem com uma confirmação após o armazenamento local.

A implementação foi desenvolvida utilizando **sockets TCP** para todas as comunicações. O sistema foi projetado para suportar M clientes distintos, onde cada cliente é identificado por um ID e possui seus próprios arquivos replicados, armazenados em diretórios separados (exemplo: `files_server/ID/`).

2. Protocolo de Comunicação

O protocolo de comunicação é baseado em mensagens de tamanho fixo, sendo o **tamanho máximo do buffer (TAM MAX) de 1024 bytes**. Cada mensagem segue a seguinte estrutura: 1 byte para o Código (indicando o tipo de mensagem) e 1023 bytes para o Payload. Funções auxiliares como `sendMessage` e `rcvMessage` gerenciam a transmissão e recepção, além de verificar códigos de erro (ERROR).

2.1. Formatos Chave de Mensagens

- **Mensagem de ID (ID):** Usada para identificar o cliente (ou servidor primário se comunicando com a réplica). O formato inclui 1 byte para o Código, 1 byte para o Tamanho do ID, e n bytes para o ID em si. O ID do cliente deve ser validado para ter no máximo 255 caracteres e conter apenas letras, números, hífen (-) ou sublinhado (_).

- **Mensagem de Metadados de Arquivo (FILEINFO):** Essencial para o processo de `upload`. É enviada antes do conteúdo do arquivo e contém 1 byte para o Código, 8 bytes para o Tamanho do Arquivo, 1 byte para o Tamanho do Nome e n bytes para o Nome do Arquivo.
- **Mensagem de Conteúdo (DATA):** Usada para transmitir os blocos do arquivo, sendo 1 byte para o Código e 1023 bytes para os dados brutos.
- **Mensagem de Resposta de Lista (LISTASW):** Usada pelo servidor para enviar a listagem de arquivos ao cliente. Contém 1 byte para o Código, 1 byte para um indicador de status (MORE, LAST ou NONE), 2 bytes para o número de nomes e os outros n bytes indicam várias vezes o tamanho do nome, e em seguida o nome em si.
- **Outras mensagens (1 byte cada):** Estas mensagens são compostas apenas pelo *código* correspondente, ocupando exatamente 1 byte. Cada uma delas possui um significado único e indica ao sistema a ação que deve ser tomada:
 - OK: Enviada para confirmar que a operação anterior foi concluída com sucesso, permitindo que o outro lado continue a comunicação.
 - LISTREQ: Mensagem usada pelo cliente para solicitar ao servidor a lista de arquivos disponíveis.
 - EXIT: Indica que o cliente deseja encerrar sua conexão, solicitando ao servidor o fechamento apenas da sessão atual.
 - END: Comando utilizado para solicitar o encerramento total do servidor, finalizando todas as conexões.
 - ERROR: Enviada pelo servidor para indicar que ocorreu algum tipo de erro durante o processamento da requisição.

3. Implementação das Funcionalidades

3.1. Conexão e Inicialização

O cliente, ao iniciar, verifica se o ID é válido, obtém as informações de endereço do servidor primário (`getaddrinfo`) e tenta estabelecer a conexão TCP (`connectToServer`). Em seguida, envia seu ID (`sendId`) e aguarda a confirmação do servidor primário.

O Servidor Primário e o Servidor Réplica utilizam o ID recebido para montar o caminho do diretório de armazenamento, garantindo que os arquivos de cada cliente fiquem separados (exemplo: `dirPath` é montado com o `srcDirPath` e o ID). A função `rcvId` é responsável por essa lógica. A criação dos diretórios é garantida pela função `createDir`.

3.2. Envio de Arquivos (upload)

Quando o cliente executa `upload <file>`, o processo segue um fluxo de três fases:

3.2.1. 1. Cliente para Servidor Primário

O cliente verifica o tamanho e a existência do arquivo (`stat`).

1. **Envio de Metadados:** O cliente envia os metadados do arquivo (`sendFileInfo`) e aguarda a confirmação do servidor.

2. **Transmissão de Dados:** O arquivo é lido e enviado em blocos (`reps`) de tamanho `TAM_MAX-1`. O servidor primário recebe o arquivo (`rcvFile`), armazena-o localmente, e envia uma confirmação (OK) ao cliente.

3.2.2. 2. Replicação para Servidores Réplica

O servidor primário, após o armazenamento local do arquivo, inicia o processo de replicação, que deve ser feito por meio de **conexões TCP individuais** com cada servidor réplica configurado. O servidor primário lê as configurações das réplicas de um arquivo e itera sobre elas. Para cada réplica:

1. **Criação de Conexão:** O servidor primário estabelece uma nova conexão (`connectToServer`) e envia o ID do cliente (`sendId`).
2. **Transmissão Completa:** A função `sendFile` é chamada, enviando os metadados (já contidos no `buffer` recebido do cliente) e o conteúdo do arquivo em blocos (`DATA`).
3. **Confirmação de Sucesso:** O servidor primário aguarda a confirmação de sucesso (OK) da réplica. Caso haja falha em qualquer etapa (ID, conexão ou envio de dados), o erro é reportado.

Uma variável `correct` é incrementada para cada réplica que confirmar o armazenamento.

3.2.3. 3. Confirmação ao Cliente

Ao final da iteração sobre todas as N réplicas, o servidor primário envia uma mensagem final (OK) ao cliente, incluindo no payload o valor de `correct` (o número de réplicas que receberam o arquivo com sucesso). O cliente exibe o resultado da replicação.

3.3. Listagem de Arquivos (`list`)

O comando `list` permite ao cliente visualizar todos os arquivos armazenados para seu ID.

1. **Solicitação:** O cliente envia uma mensagem `LISTREQ`.
2. **Processamento no Servidor:** O servidor principal executa a função `sendList`, que utiliza as funções `opendir` e `readdir` para listar os arquivos regulares (`S_ISREG`) presentes no diretório do cliente. Os nomes dos arquivos são temporariamente gerenciados por uma estrutura de lista encadeada (`node_t`).
3. **Envio da Lista:** O servidor serializa a lista de nomes. Se a lista completa exceder o tamanho do buffer, ela é enviada em múltiplas mensagens `LISTASW`, utilizando o status `MORE` para indicar que mais dados virão, e `LAST` para indicar o final. Se não houver arquivos, é enviado o status `NONE`.
4. **Cliente:** O cliente recebe e concatena os blocos de lista até receber a mensagem com o status `LAST`, exibindo o resultado na tela.

3.4. Outros Comandos

- **help:** Imprime os comandos e suas descrições na tela.
- **exit:** O cliente envia uma mensagem EXIT para fechar a conexão com o servidor primário.
- **endall:** O cliente envia uma mensagem END que instrui o servidor primário e as réplicas a fecharem seus sockets e terminarem a execução.

4. Decisões de Projeto e Estruturas Auxiliares

4.1. Gerenciamento de Dados Maiores que o Buffer

Uma decisão crítica de projeto foi o tratamento de arquivos maiores que TAM_MAX. Tanto o cliente quanto o servidor primário, ao enviarem arquivos, dividem o conteúdo em blocos de 1023 bytes (para reservar 1 byte para o código DATA) e repetem o envio (fread, sendMessage). A confiabilidade da transmissão de blocos é garantida pela exigência de uma confirmação (rcvMessage) após a transmissão de cada bloco.

4.2. Estrutura de Lista Encadeada

Para a funcionalidade de listagem de arquivos, foi implementada uma estrutura de lista encadeada (node_t). Esta estrutura armazena o conteúdo (content) e o tamanho (size) de cada nome de arquivo, facilitando a iteração e a serialização dos nomes antes do envio ao cliente (função sendList). A função destroyList garante a liberação de memória após o uso.

5. Modo de uso

Compilação:

```
$ make
```

Primeiramente, executar cada uma das réplicas com:

```
$ ./mirror/mirror <porta> <dir>
```

Depois, escrever um arquivo de configuração no seguinte padrão:

```
<número n de réplicas>
<IP da réplica 1> <porta da réplica 1>
<IP da réplica 2> <porta da réplica 2>
...
<IP da réplica n> <porta da réplica n>
```

Executar o servidor:

```
$ ./server/server <porta> <arquivo de configuração>
```

Executar o cliente:

```
$ ./client/client <id> <host> <porta>
```

Para esses exemplos funcionarem, é necessário o usuário estar na pasta pai do sistema.